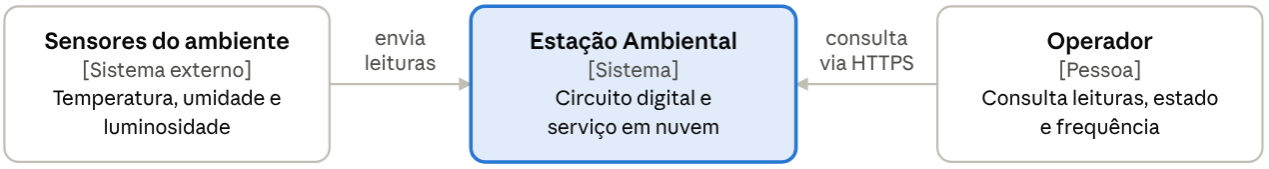
Arquitetura da Estação Ambiental (Modelo C4)

Projeto INTERFACE, ExpoTech 2026.2. Este documento descreve a arquitetura em três níveis do Modelo C4: **contexto**, **contêineres** e **componentes**. Cada elemento marcado como *planejado* ainda não existe e será entregue na etapa de nuvem (Azure). Tudo o que não tem essa marca já está implementado e testado.

Nível 1: Contexto do sistema

A Estação Ambiental mede temperatura, umidade e luminosidade de um ambiente, decide se está tudo normal, em atenção ou em alarme, guarda o histórico na nuvem e calcula com que frequência cada sensor deve ler para gastar menos energia.

A Estação Ambiental une sensores e operador em um único sistema



O circuito decide o LED localmente; o serviço registra o estado na nuvem.

contexto do sistema · 3 elementos

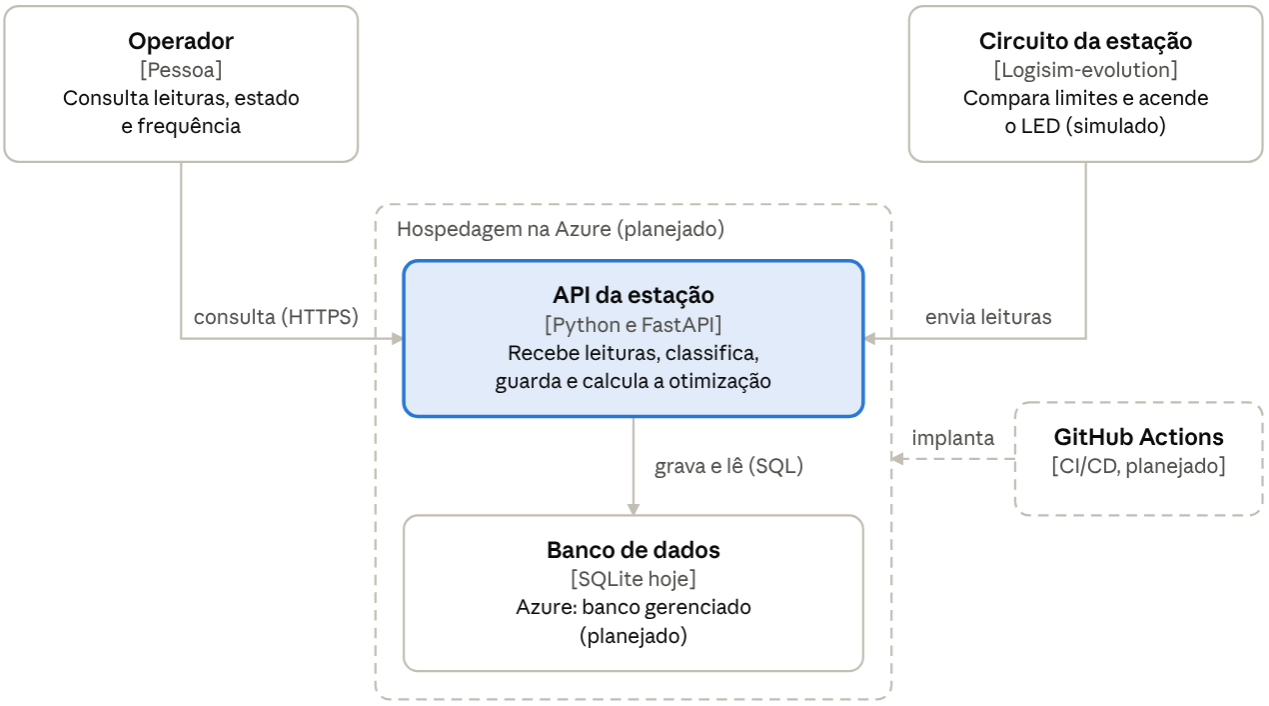
Os sensores fornecem as leituras e o operador consulta o estado e a frequência recomendada pela interface da API.

Elemento	Tipo	Papel
Operador	Pessoa	Consulta as leituras, o estado e a frequência recomendada pela interface da API
Sensores do ambiente	Sistema externo	Fornecem temperatura, umidade e luminosidade
Estação Ambiental	Sistema (nosso)	Circuito digital mais serviço em nuvem

Nível 2: Contêineres

O sistema se divide em quatro partes que rodam separadamente. O ponto de contato entre o circuito físico e o serviço em nuvem é o envio das leituras pela API.

O circuito envia leituras à API, que classifica e guarda no banco



Linha tracejada: ainda não existe (planejado).

contêineres · 4 elementos

O circuito envia leituras à API e o operador a consulta; a API grava no banco.

Contêiner	Tecnologia	Responsabilidade	Situação
Circuito da estação	Logisim-evolution	Compara cada leitura com os limites e acende o LED normal, atenção ou alarme	Pronto (simulado)
API da estação	Python e FastAPI	Recebe leituras, classifica, guarda e calcula a otimização	Pronta e testada
Banco de dados	SQLite hoje; banco gerenciado da Azure depois	Guarda o histórico de leituras	SQLite pronto; Azure planejado

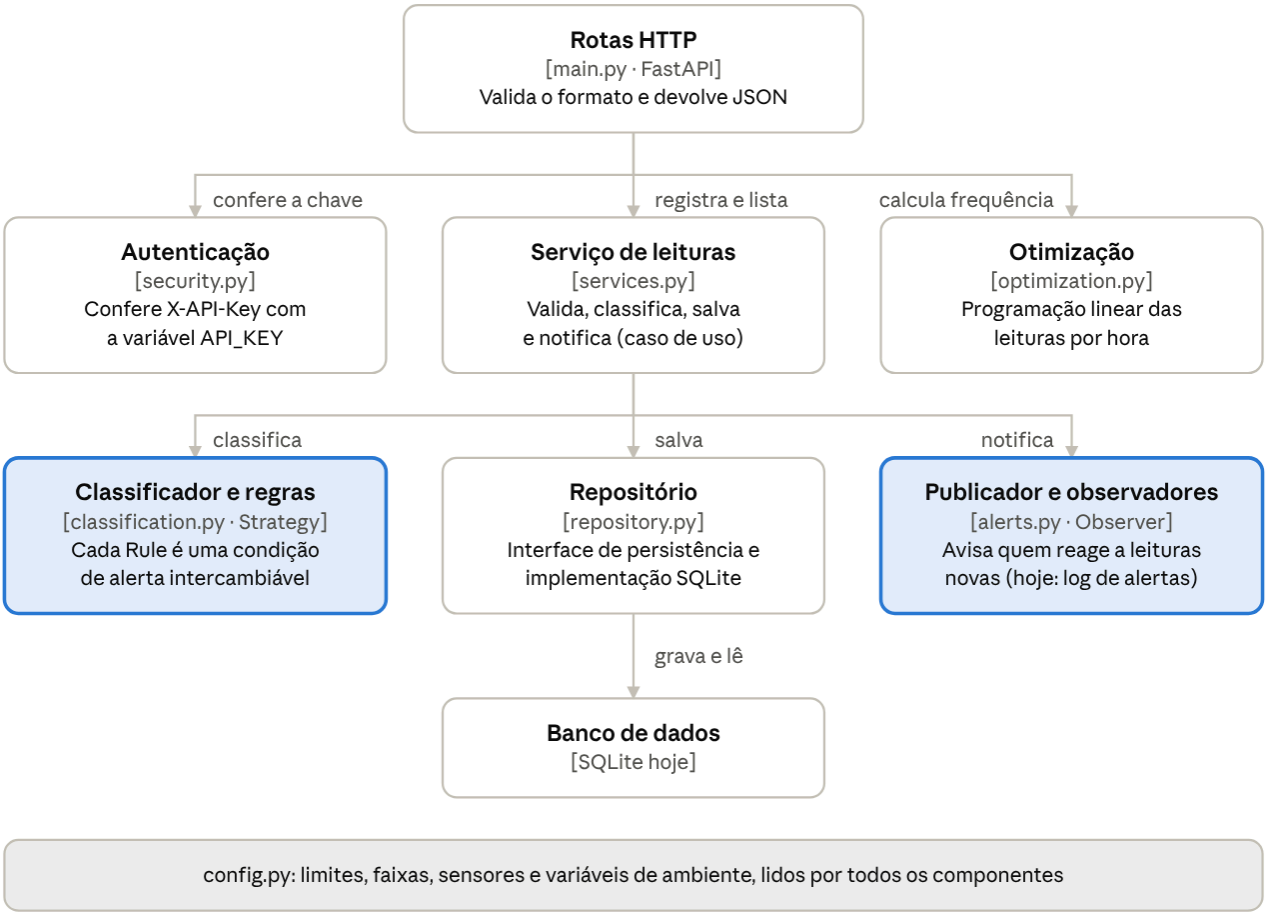
Contêiner	Tecnologia	Responsabilidade	Situação
Hospedagem e entrega	Azure App Service e GitHub Actions	Coloca a API no ar e roda testes e implantação a cada mudança	Planejado

Como os dados circulam: o circuito decide localmente qual LED acender. A API aplica a mesma regra (limites iguais aos do circuito) para registrar o estado no histórico. Na estação real, um microcontrolador enviaria cada leitura para a API por HTTPS (método POST, com o cabeçalho `X-API-Key`). Na simulação do projeto, a ligação entre o Logisim e a API é feita enviando os mesmos valores à API.

Nível 3: Componentes da API

A API é organizada em camadas. Somente `main.py` conhece HTTP; as regras ficam em módulos independentes.

Strategy e Observer ficam isolados do HTTP e do banco



componentes da API · 8 componentes

As rotas chamam o serviço, que usa o classificador, o repositório e o publicador. Os dois componentes destacados são os padrões GoF do projeto.

Componente	Arquivo	Responsabilidade
Rotas HTTP	`main.py`	Recebe as requisições, valida o formato e devolve JSON; é o ponto onde as peças são ligadas
Autenticação	`security.py`	Confere a chave `X-API-Key` com a variável de ambiente `API_KEY`
Serviço de leituras	`services.py`	Valida, classifica, salva e notifica (caso de uso)
Classificador e regras	`classification.py`	Padrão Strategy: cada regra é uma condição de alerta
Publicador e observadores	`alerts.py`	Padrão Observer: avisa quem reage a leituras novas (hoje, o log de alertas)
Repositório	`repository.py`	Interface de persistência e implementação em SQLite
Otimização	`optimization.py`	Modelo de programação linear que define as leituras por hora de cada sensor
Configuração	`config.py`	Limites, faixas, parâmetros dos sensores e variáveis de ambiente

Interface da API

Método e rota	Autenticação	Função
`GET /health`	Não	Verifica se a API está no ar
`POST /readings`	Cabeçalho `X-API-Key`	Registra uma leitura e devolve o estado (`desligado` , `normal` , `atencao` ou `alarme`)
`GET /readings?limit=50`	Não	Lista as últimas leituras, a mais recente primeiro
`GET /optimization/schedule?orcamento_mj_hora=1000`	Não	Devolve as leituras por hora e o intervalo de cada sensor

Decisões de arquitetura

1. **FastAPI e Python.** Cria a API com pouco código e gera sozinho a tela de testes (`/docs`), que serve de interface para demonstrar o sistema.
2. **Camadas com repositório por interface.** O banco pode ser trocado (SQLite para banco gerenciado da Azure) sem alterar rotas nem regras.
3. **Chave de API na escrita.** Somente quem tem a chave grava leituras. A chave vem de variável de ambiente e nunca fica no código.
4. **Mesma regra no circuito e na API.** Os limites são iguais (temperatura acima de 30, umidade abaixo de 30, luminosidade acima de 80), e os testes comprovam que as duas implementações dão o mesmo resultado.
5. **Logs como base de observabilidade.** O observador de alertas escreve no log, que será consultado no fluxo de logs da Azure.

O que ainda não existe (planejado)

- Banco de dados gerenciado da Azure no lugar do SQLite.
- API publicada no Azure App Service, com `API_KEY` e a conexão do banco em variáveis de ambiente.
- Pipeline de CI/CD no GitHub Actions (testes e implantação).
- Microcontrolador real enviando leituras (a simulação do projeto substitui essa etapa).